

# ESTRUTURA DE DADOS NÃO-LINEARES

## Implementação de Árvore AVL com Herança de ABP

Aluna: Larissa Samara | Professor: Robinson Alves | Material: *árvoreAVL.pdf*

### 1. Fundamentação Teórica e Fórmulas de Aula (Prof. Robinson Alves)

Conforme apresentado nos slides do Prof. Robinson Alves (*árvoreAVL.pdf*), a Árvore AVL mantém o equilíbrio rigoroso através do Fator de Balanceamento (FB) calculado para cada nó  $v$  como:

**Definição do Fator de Balanceamento (Slide do Prof. Robinson):**  $FB(v) = he(v) - hd(v) + 1$  : subárvore esquerda mais alta que a direita 0 : subárvore esquerda igual a direita -1 : subárvore direita mais alta do que a esquerda Valores balanceados: -1, 0 ou 1. Nós desregulados:  $FB > 1$  ou  $FB < -1$ .

A tabela a seguir resume as variações de FB e os critérios de parada dos antecessores ensinados na aula:

**Critérios de Parada na Propagação do FB:** Inserção: ArvEsq (+1), ArvDir (-1) -> Regra: 'Se  $FB(Vantecessor) == 0$  pare' Remoção: ArvEsq (-1), ArvDir (+1) -> Regra: 'Se  $FB(Vantecessor) != 0$  pare'

Para a reestruturação e recálculo em  $O(1)$  dos fatores de balanceamento dos nós B (raiz original da rotação) e A (novo topo da subárvore), o código adota as fórmulas dos slides (baseadas em StackExchange CS #48861):

**Rotação Esquerda Simples (RES):**  $FB\_B\_novo = FB\_B + 1 - \min(FB\_A, 0)$ ;  $FB\_A\_novo = FB\_A + 1 + \max(FB\_B\_novo, 0)$ ;

**Rotação Simples a Direita (RSD):**  $FB\_B\_novo = FB\_B - 1 - \max(FB\_A, 0)$ ;  $FB\_A\_novo = FB\_A - 1 + \min(FB\_B\_novo, 0)$ ;

As rotações duplas são executadas conforme os passos descritos no slide: - Rotação Dupla a Esquerda (RDE): RSD na subárvore direita do nó desbalanceado, seguida de RES no nó desbalanceado. - Rotação Dupla a Direita (RDD): RES na subárvore esquerda do nó desbalanceado, seguida de RSD no nó desbalanceado.

### 2. Detalhamento do Exemplo da Especificação

#### 2.1 Inserção do Nó 25 e Aplicação da RES em 15

Após inserir 10, 5, 15, 2, 8, 22, a árvore possui o nó 15 com  $FB = -1$  devido ao filho direito 22. Ao inserir 25: 1. 25 é alocado à direita de 22. O nó 22 passa para  $FB = -1$ . 2. O aumento propaga para o nó 15: seu FB passa de -1 para -2 (desbalanceamento à direita). 3. Regra do slide: com  $FB = -2$  e subárvore direita com  $FB \leq 0$  ( $FB(22) = -1$ ), aplica-se a Rotação Esquerda Simples (RES) no nó 15. 4. Fórmulas de recálculo:  $FB\_B\_novo$  (nó 15) =  $-2 + 1 - \min(-1, 0) = -2 + 1 - (-1) = 0$   $FB\_A\_novo$  (nó 22) =  $-1 + 1 + \max(0, 0) = -1 + 1 + 0 = 0$  5. O nó 22 torna-se nova raiz com 15 à esquerda e 25 à direita, ambos com  $FB = 0$ .

## 2.2 Remoção do Nó 5

O nó 5 tem dois filhos (2 e 8). Pela regra de remoção de ABP/AVL: 1. Substitui 5 pelo seu sucessor in-order (menor nó da subárvore direita), que é 8. 2. Remove a folha original 8 da subárvore direita. 3. Ao remover da subárvore direita de 8, calcula-se  $FB(8): he(2) - hd(vazio) = 1 - 0 = +1$ . 4. Regra de parada do slide do Prof. Robinson: 'Se  $FB(Vantecessor) \neq 0$  pare'. 5. Como  $FB(8) = +1 \neq 0$ , a diminuição de altura não se propaga para a raiz 10, permanecendo  $FB(10) = 0$ .

## 3. Código-Fonte Completo em Java

O código-fonte foi desenvolvido utilizando herança estrita (ArvoreAVL estende ArvoreBinariaPesquisa, e NoAVL estende NoABP). Abaixo está o código completo formatado em fonte monoespaçada (Consolas 9.5pt) com o padrão de cores da IDE Eclipse.

### Arquivo: NoABP.java

```
/**
 * Classe que representa um nó na Árvore Binária de Pesquisa (ABP).
 * Serve como classe base para os nós da Árvore AVL através de herança.
 */
@author Larissa Samara
*/
public class NoABP {
    public int chave;
    public NoABP esquerdo;
    public NoABP direito;

    /**
     * Construtor do nó básico de ABP.
     * @param chave Valor inteiro armazenado no nó.
     */
    public NoABP(int chave) {
        this.chave = chave;
        this.esquerdo = null;
        this.direito = null;
    }
}
```

### Arquivo: ArvoreBinariaPesquisa.java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Implementação da Árvore Binária de Pesquisa (ABP).
 * Fornece as operações fundamentais (busca, inserção, remoção, exibição)
 * e serve como superclasse para a ArvoreAVL.
 */
@author Larissa Samara
*/
public class ArvoreBinariaPesquisa {
    protected NoABP raiz;

    public ArvoreBinariaPesquisa() {
        this.raiz = null;
    }

    public NoABP getRaiz() {
        return this.raiz;
    }
}
```

```

public boolean estaVazia() {
    return this.raiz == null;
}

/**
 * Busca por uma chave na árvore.
 * Complexidade de tempo:  $O(h)$ , sendo  $O(\log n)$  se balanceada.
 * @param chave Valor a ser buscado.
 * @return true se a chave existe na árvore, false caso contrário.
 */
public boolean buscar(int chave) {
    return buscarRec(this.raiz, chave) != null;
}

protected NoABP buscarRec(NoABP atual, int chave) {
    if (atual == null || atual.chave == chave) {
        return atual;
    }
    if (chave < atual.chave) {
        return buscarRec(atual.esquerdo, chave);
    } else {
        return buscarRec(atual.direito, chave);
    }
}

/**
 * Inserção padrão de ABP.
 * @param chave Valor a ser inserido.
 */
public void inserir(int chave) {
    this.raiz = inserirRec(this.raiz, chave);
}

protected NoABP inserirRec(NoABP atual, int chave) {
    if (atual == null) {
        return new NoABP(chave);
    }
    if (chave < atual.chave) {
        atual.esquerdo = inserirRec(atual.esquerdo, chave);
    } else if (chave > atual.chave) {
        atual.direito = inserirRec(atual.direito, chave);
    }
    return atual;
}

/**
 * Remoção padrão de ABP.
 * @param chave Valor a ser removido.
 */
public void remover(int chave) {
    this.raiz = removerRec(this.raiz, chave);
}

protected NoABP removerRec(NoABP atual, int chave) {
    if (atual == null) {
        return null;
    }

    if (chave < atual.chave) {
        atual.esquerdo = removerRec(atual.esquerdo, chave);
    } else if (chave > atual.chave) {
        atual.direito = removerRec(atual.direito, chave);
    } else {

```

```

// Caso 1 e Caso 2: nó folha ou com apenas 1 filho
if (atual.esquerdo == null) {
    return atual.direito;
} else if (atual.direito == null) {
    return atual.esquerdo;
}

// Caso 3: nó com 2 filhos
// Substitui pelo sucessor (menor elemento da subárvore direita)
NoABP sucessor = obterMinimo(atual.direito);
atual.chave = sucessor.chave;
atual.direito = removerRec(atual.direito, sucessor.chave);
}
return atual;
}

/**
 * Encontra o nó com menor valor na subárvore.
 */
protected NoABP obterMinimo(NoABP atual) {
    NoABP temp = atual;
    while (temp != null && temp.esquerdo != null) {
        temp = temp.esquerdo;
    }
    return temp;
}

/**
 * Exibe a árvore visualmente por níveis, alinhando as colunas
 * de acordo com a ordem simétrica (in-order).
 */
public void mostrar() {
    mostrarFormatado(this.raiz);
}

protected void mostrarFormatado(NoABP raizSubarvore) {
    if (raizSubarvore == null) {
        System.out.println(" (Árvore vazia)");
        return;
    }

    Map<NoABP, Integer> xPos = new HashMap<>();
    Map<NoABP, Integer> yPos = new HashMap<>();
    int[] col = new int[]{0};
    calcularPosicoes(raizSubarvore, 0, col, xPos, yPos);

    int profundidadeMaxima = 0;
    for (int d : yPos.values()) {
        if (d > profundidadeMaxima) {
            profundidadeMaxima = d;
        }
    }

    int larguraColuna = 10;
    for (int d = 0; d <= profundidadeMaxima; d++) {
        StringBuilder linha = new StringBuilder();
        List<NoABP> nosDoNivel = new ArrayList<>();
        for (Map.Entry<NoABP, Integer> entry : yPos.entrySet()) {
            if (entry.getValue() == d) {
                nosDoNivel.add(entry.getKey());
            }
        }
        nosDoNivel.sort(Comparator.comparingInt(xPos::get));
    }
}

```

```

        for (NoABP n : nosDoNivel) {
            int colunaAlvo = xPos.get(n) * larguraColuna + 4;
            while (linha.length() < colunaAlvo) {
                linha.append(" ");
            }
            String rotulo;
            if (n instanceof NoAVL) {
                rotulo = n.chave + "[" + ((NoAVL) n).fb + "]";
            } else {
                rotulo = String.valueOf(n.chave);
            }
            linha.append(rotulo);
        }
        System.out.println(linha.toString());
    }
}

private void calcularPosicoes(NoABP n, int profundidade, int[] col, Map<NoABP, Integer> xPos, Map<NoABP, Integer> yPos) {
    if (n == null) return;
    calcularPosicoes(n.esquerdo, profundidade + 1, col, xPos, yPos);
    xPos.put(n, col[0]++);
    yPos.put(n, profundidade);
    calcularPosicoes(n.direito, profundidade + 1, col, xPos, yPos);
}
}

```

### Arquivo: NoAVL.java

```

/**
 * Classe que representa um nó na Árvore AVL, estendendo NoABP por herança.
 * Adiciona o atributo de Fator de Balanceamento (FB).
 *
 * Convenção adotada na disciplina:
 * FB(nó) = altura(subárvore esquerda) - altura(subárvore direita)
 *
 * Valores possíveis para nós balanceados em AVL: -1, 0, +1.
 * Desbalanceamento ocorre quando FB atinge +2 ou -2.
 *
 * @author Larissa Samara
 */

```

```

public class NoAVL extends NoABP {
    public int fb;

```

```

    /**
     * Construtor do nó AVL.
     * @param chave Valor inteiro a ser armazenado.
     */
    public NoAVL(int chave) {
        super(chave);
        this.fb = 0;
    }
}

```

### Arquivo: ArvoreAVL.java

```

/**
 * Implementação da Árvore AVL através de herança de ArvoreBinariaPesquisa (ABP),
 * estritamente alinhada ao material e slides do Prof. Robinson Alves (árvoreAVL.pdf).
 *
 * Características principais:
 * 1. Herança direta de ArvoreBinariaPesquisa.
 * 2. Operações básicas em  $O(\log n)$ .
 * 3. Fator de Balanceamento:
 *     $FB(v) = he(v) - hd(v)$ 

```

```

* +1: subárvore esquerda mais alta que a direita
* 0: subárvore esquerda igual a direita
* -1: subárvore direita mais alta do que a esquerda
* 4. Fórmulas exatas de recálculo dos slides do Prof. Robinson Alves:
* - Rotação Esquerda Simples (RES):
*    $FB\_B\_novo = FB\_B + 1 - \min(FB\_A, 0);$ 
*    $FB\_A\_novo = FB\_A + 1 + \max(FB\_B\_novo, 0);$ 
* - Rotação Simples a Direita (RSD):
*    $FB\_B\_novo = FB\_B - 1 - \max(FB\_A, 0);$ 
*    $FB\_A\_novo = FB\_A - 1 + \min(FB\_B\_novo, 0);$ 
* 5. Critérios de parada na atualização do FB dos antecessores:
* - Inserção: "Se  $FB(Vantecessor) == 0$  pare"
* - Remoção: "Se  $FB(Vantecessor) != 0$  pare"
*
* @author Larissa Samara
*/

```

```

public class ArvoreAVL extends ArvoreBinariaPesquisa {

    // Flag que indica se a altura da subárvore variou durante a recursão
    private boolean mudouAltura;

    public ArvoreAVL() {
        super();
    }

    /**
     * Inserção com balanceamento AVL em  $O(\log n)$ .
     * @param chave Valor a ser inserido.
     */
    @Override
    public void inserir(int chave) {
        this.mudouAltura = false;
        this.raiz = inserirAVL((NoAVL) this.raiz, chave);
    }

    private NoAVL inserirAVL(NoAVL atual, int chave) {
        if (atual == null) {
            this.mudouAltura = true;
            return new NoAVL(chave);
        }

        if (chave < atual.chave) {
            atual.esquerdo = inserirAVL((NoAVL) atual.esquerdo, chave);
            if (this.mudouAltura) {
                // Inserção na ArvEsq: soma +1 ao FB conforme tabela do slide
                switch (atual.fb) {
                    case -1:
                        atual.fb = 0;
                        this.mudouAltura = false; // "Se  $FB(Vantecessor) == 0$  pare"
                        break;
                    case 0:
                        atual.fb = 1;
                        this.mudouAltura = true; // Continua propagando
                        break;
                    case 1:
                        atual.fb = 2; // Desbalanceou à esquerda!
                        atual = balancearEsquerda(atual);
                        this.mudouAltura = false; // Após rotação na inserção, altura volta à original: pare
                        break;
                }
            }
        } else if (chave > atual.chave) {
            atual.direito = inserirAVL((NoAVL) atual.direito, chave);

```

```

    if (this.mudouAltura) {
        // Inserção na ArvDir: subtrai -1 do FB conforme tabela do slide
        switch (atual.fb) {
            case 1:
                atual.fb = 0;
                this.mudouAltura = false; // "Se FB(Vantecedor) == 0 pare"
                break;
            case 0:
                atual.fb = -1;
                this.mudouAltura = true; // Continua propagando
                break;
            case -1:
                atual.fb = -2; // Desbalanceou à direita!
                atual = balancearDireita(atual);
                this.mudouAltura = false; // Após rotação na inserção, altura volta à original: pare
                break;
        }
    }
} else {
    // Chave duplicada: não permite duplicatas na árvore
    this.mudouAltura = false;
}

return atual;
}

/**
 * Trata o desbalanceamento após inserção ou remoção na subárvore esquerda (FB = +2).
 * Regra do slide do Prof. Robinson Alves:
 * - Se FB >= 0 na subárvore esquerda: Rotação Simples a Direita (RSD)
 * - Se FB < 0 na subárvore esquerda: Rotação Dupla a Direita (RDD)
 */
private NoAVL balancearEsquerda(NoAVL pivo) {
    NoAVL filhoEsq = (NoAVL) pivo.esquerdo;
    if (filhoEsq.fb >= 0) {
        return rotacaoDireitaSimples(pivo);
    } else {
        return rotacaoDuplaDireita(pivo);
    }
}

/**
 * Trata o desbalanceamento após inserção ou remoção na subárvore direita (FB = -2).
 * Regra do slide do Prof. Robinson Alves:
 * - Se FB <= 0 na subárvore direita: Rotação Esquerda Simples (RES)
 * - Se FB > 0 na subárvore direita: Rotação Dupla a Esquerda (RDE)
 */
private NoAVL balancearDireita(NoAVL pivo) {
    NoAVL filhoDir = (NoAVL) pivo.direito;
    if (filhoDir.fb <= 0) {
        return rotacaoEsquerdaSimples(pivo);
    } else {
        return rotacaoDuplaEsquerda(pivo);
    }
}

/**
 * Rotação Esquerda Simples (RES).
 * Conforme os slides do Prof. Robinson Alves:
 *
 * Estrutura inicial:
 *   B (desbalanceado, FB = -2)
 *   |

```

```

*      A (filho direito, FB <= 0)
*      /\
*      T2 T3
*
* Passos do slide:
* 1. Guarde a subárvore direita (A).
* 2. Troque a subárvore guardada pela subárvore esquerda da árvore guardada (B.direito = A.esquerdo).
* 3. Ponha na subárvore esquerda da subárvore guardada a árvore restante (A.esquerdo = B).
* 4. Atualize o FB pelas fórmulas dos slides:
*      FB_B_novo = FB_B + 1 - min(FB_A, 0);
*      FB_A_novo = FB_A + 1 + max(FB_B_novo, 0);
*
* @param B Nó desbalanceado (pivô original)
* @return Nova raiz da subárvore (A)
*/
public NoAVL rotacaoEsquerdaSimples(NoAVL B) {
    NoAVL A = (NoAVL) B.direito;
    B.direito = A.esquerdo;
    A.esquerdo = B;

    // Fórmulas exatas do slide do Prof. Robinson Alves:
    int FB_B_novo = B.fb + 1 - Math.min(A.fb, 0);
    int FB_A_novo = A.fb + 1 + Math.max(FB_B_novo, 0);

    B.fb = FB_B_novo;
    A.fb = FB_A_novo;

    return A;
}

/**
* Rotação Simples a Direita (RSD).
* Conforme os slides do Prof. Robinson Alves (simétrica à RES):
*
* Estrutura inicial:
*      B (desbalanceado, FB = +2)
*      /
*      A (filho esquerdo, FB >= 0)
*      /\
*      T1 T2
*
* Fórmulas do slide:
*      FB_B_novo = FB_B - 1 - max(FB_A, 0);
*      FB_A_novo = FB_A - 1 + min(FB_B_novo, 0);
*
* @param B Nó desbalanceado (pivô original)
* @return Nova raiz da subárvore (A)
*/
public NoAVL rotacaoDireitaSimples(NoAVL B) {
    NoAVL A = (NoAVL) B.esquerdo;
    B.esquerdo = A.direito;
    A.direito = B;

    // Fórmulas exatas do slide do Prof. Robinson Alves:
    int FB_B_novo = B.fb - 1 - Math.max(A.fb, 0);
    int FB_A_novo = A.fb - 1 + Math.min(FB_B_novo, 0);

    B.fb = FB_B_novo;
    A.fb = FB_A_novo;

    return A;
}

```

```

/**
 * Rotação Dupla a Esquerda (RDE).
 * Passos conforme o slide do Prof. Robinson Alves:
 * 1. Efetua-se uma rotação simples direita na subárvore direita do nó desbalanceado (RSD).
 * 2. Realiza-se uma rotação simples esquerda no nó desbalanceado (RES).
 *
 * @param B Nó desbalanceado
 * @return Nova raiz da subárvore
 */
public NoAVL rotacaoDuplaEsquerda(NoAVL B) {
    B.direito = rotacaoDireitaSimples((NoAVL) B.direito);
    return rotacaoEsquerdaSimples(B);
}

/**
 * Rotação Dupla a Direita (RDD).
 * Passos conforme o slide do Prof. Robinson Alves:
 * 1. Efetuar uma rotação simples esquerda na subárvore esquerda do nó desbalanceado (RES).
 * 2. Realizar uma rotação simples direita no nó desbalanceado (RSD).
 *
 * @param B Nó desbalanceado
 * @return Nova raiz da subárvore
 */
public NoAVL rotacaoDuplaDireita(NoAVL B) {
    B.esquerdo = rotacaoEsquerdaSimples((NoAVL) B.esquerdo);
    return rotacaoDireitaSimples(B);
}

// Aliases utilizando os acrônimos dos slides (RES, RSD, RDE, RDD)
public NoAVL RES(NoAVL B) { return rotacaoEsquerdaSimples(B); }
public NoAVL RSD(NoAVL B) { return rotacaoDireitaSimples(B); }
public NoAVL RDE(NoAVL B) { return rotacaoDuplaEsquerda(B); }
public NoAVL RDD(NoAVL B) { return rotacaoDuplaDireita(B); }

// Aliases para compatibilidade de nomenclaturas
public NoAVL rotacaoSimplesEsquerda(NoAVL B) { return rotacaoEsquerdaSimples(B); }
public NoAVL rotacaoSimplesDireita(NoAVL B) { return rotacaoDireitaSimples(B); }

/**
 * Remoção com balanceamento AVL em  $O(\log n)$ .
 * @param chave Valor a ser removido.
 */
@Override
public void remover(int chave) {
    this.mudouAltura = false;
    this.raiz = removerAVL((NoAVL) this.raiz, chave);
}

private NoAVL removerAVL(NoAVL atual, int chave) {
    if (atual == null) {
        this.mudouAltura = false;
        return null;
    }

    if (chave < atual.chave) {
        atual.esquerdo = removerAVL((NoAVL) atual.esquerdo, chave);
        if (this.mudouAltura) {
            // Remoção na ArvEsq: subtrai -1 do FB conforme tabela do slide
            atual = aposRemocaoEsquerda(atual);
        }
    } else if (chave > atual.chave) {
        atual.direito = removerAVL((NoAVL) atual.direito, chave);
        if (this.mudouAltura) {

```

```

        // Remoção na ArvDir: soma +1 ao FB conforme tabela do slide
        atual = aposRemocaoDireita(atual);
    }
} else {
    // Encontrou o nó a ser removido
    if (atual.esquerdo == null) {
        this.mudouAltura = true;
        return (NoAVL) atual.direito;
    } else if (atual.direito == null) {
        this.mudouAltura = true;
        return (NoAVL) atual.esquerdo;
    } else {
        // Nó com 2 filhos: substitui pelo sucessor (menor da subárvore direita)
        NoAVL sucessor = (NoAVL) obterMinimo(atual.direito);
        atual.chave = sucessor.chave;
        atual.direito = removerAVL((NoAVL) atual.direito, sucessor.chave);
        if (this.mudouAltura) {
            atual = aposRemocaoDireita(atual);
        }
    }
}
}
return atual;
}

/**
 * Trata o ajuste de FB e eventuais rotações quando a subárvore esquerda diminui de altura.
 * Conforme tabela do slide: ArvEsq -1, "Se FB(Vantecessor) != 0 pare".
 */
private NoAVL aposRemocaoEsquerda(NoAVL atual) {
    switch (atual.fb) {
        case 1:
            atual.fb = 0;
            this.mudouAltura = true; // Altura diminuiu, continua propagando
            break;
        case 0:
            atual.fb = -1;
            this.mudouAltura = false; // "Se FB(Vantecessor) != 0 pare"
            break;
        case -1:
            atual.fb = -2; // Desbalanceou à direita!
            NoAVL dir = (NoAVL) atual.direito;
            if (dir.fb <= 0) {
                // FB <= 0 na subárvore direita: RES
                atual = rotacaoEsquerdaSimples(atual);
                this.mudouAltura = (atual.fb == 0); // Se virou 0 encolheu; se != 0 pare
            } else {
                // FB > 0 na subárvore direita: RDE
                atual = rotacaoDuplaEsquerda(atual);
                this.mudouAltura = true;
            }
            break;
    }
    return atual;
}

/**
 * Trata o ajuste de FB e eventuais rotações quando a subárvore direita diminui de altura.
 * Conforme tabela do slide: ArvDir +1, "Se FB(Vantecessor) != 0 pare".
 */
private NoAVL aposRemocaoDireita(NoAVL atual) {
    switch (atual.fb) {
        case -1:
            atual.fb = 0;

```

```

        this.mudouAltura = true; // Altura diminuiu, continua propagando
        break;
    case 0:
        atual.fb = 1;
        this.mudouAltura = false; // "Se FB(Vantecedor) != 0 pare"
        break;
    case 1:
        atual.fb = 2; // Desbalanceou à esquerda!
        NoAVL esq = (NoAVL) atual.esquerdo;
        if (esq.fb >= 0) {
            // FB >= 0 na subárvore esquerda: RSD
            atual = rotacaoDireitaSimples(atual);
            this.mudouAltura = (atual.fb == 0); // Se virou 0 encolheu; se != 0 pare
        } else {
            // FB < 0 na subárvore esquerda: RDD
            atual = rotacaoDuplaDireita(atual);
            this.mudouAltura = true;
        }
        break;
    }
    return atual;
}
}

```

## Arquivo: Main.java

```
import java.util.Scanner;
```

```

/**
 * Classe principal de teste para a Árvore AVL.
 *
 * Totalmente alinhada ao material e slides do Prof. Robinson Alves (árvoreAVL.pdf).
 *
 * Funcionalidades interativas via terminal:
 * - Inclusão de nós
 * - Remoção de nós
 * - Busca de nós
 * - Visualização da árvore com chaves e Fatores de Balanceamento [FB]
 * - Demonstração passo a passo da especificação da tarefa
 * - Execução dos exercícios práticos dos slides do Prof. Robinson Alves (Slide 38 e Slide 42)
 *
 * @author Larissa Samara
 */

```

```
public class Main {
```

```

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        ArvoreAVL avl = new ArvoreAVL();
        int opcao = -1;

```

```

        System.out.println("=====
");

```

```

        System.out.println("          SISTEMA DE TESTES - ÁRVORE AVL (HERANÇA DE ABP)          ");
        System.out.println("          Conforme Slides e Aulas do Prof. Robinson Alves          ");
        System.out.println("          Aluna: Larissa Samara          ");

```

```

        System.out.println("=====
");

```

```

        while (opcao != 0) {
            System.out.println("\n----- MENU -----");
            System.out.println(" 1 - Inserir nó");
            System.out.println(" 2 - Remover nó");
            System.out.println(" 3 - Buscar nó");

```

```

System.out.println(" 4 - Mostrar árvore (com chaves e FB)");
System.out.println(" 5 - Executar exemplo da especificação (10, 5, 15, 2, 8, 22 -> 25 -> -5)");
System.out.println(" 6 - Executar exercício do Slide 38 (Inserir: 10, 20..90)");
System.out.println(" 7 - Executar exercício do Slide 42 (Remover: 40, 25, 50..60)");
System.out.println(" 8 - Limpar / Reiniciar árvore");
System.out.println(" 0 - Sair");
System.out.print("Escolha uma opção: ");

try {
    String entrada = scanner.nextLine().trim();
    if (entrada.isEmpty()) continue;
    opcao = Integer.parseInt(entrada);
} catch (NumberFormatException e) {
    System.out.println(">> Opção inválida! Digite um número inteiro.");
    continue;
}

switch (opcao) {
    case 1:
        System.out.print("Digite o valor inteiro para inserir: ");
        try {
            int valor = Integer.parseInt(scanner.nextLine().trim());
            if (avl.buscar(valor)) {
                System.out.println(">> A chave " + valor + " já existe na árvore (duplicatas não permitidas).");
            } else {
                avl.inserir(valor);
                System.out.println(">> Valor " + valor + " inserido com sucesso!");
                System.out.println("\nÁrvore atual:");
                avl.mostrar();
            }
        } catch (NumberFormatException e) {
            System.out.println(">> Valor inválido! Digite um número inteiro.");
        }
        break;

    case 2:
        System.out.print("Digite o valor inteiro para remover: ");
        try {
            int valor = Integer.parseInt(scanner.nextLine().trim());
            if (!avl.buscar(valor)) {
                System.out.println(">> A chave " + valor + " não foi encontrada na árvore.");
            } else {
                avl.remover(valor);
                System.out.println(">> Valor " + valor + " removido com sucesso!");
                System.out.println("\nÁrvore atual:");
                avl.mostrar();
            }
        } catch (NumberFormatException e) {
            System.out.println(">> Valor inválido! Digite um número inteiro.");
        }
        break;

    case 3:
        System.out.print("Digite o valor inteiro para buscar: ");
        try {
            int valor = Integer.parseInt(scanner.nextLine().trim());
            boolean achou = avl.buscar(valor);
            if (achou) {
                System.out.println(">> CHAVE ENCONTRADA: " + valor + " está presente na árvore.");
            } else {
                System.out.println(">> CHAVE NÃO ENCONTRADA: " + valor + " não existe na árvore.");
            }
        } catch (NumberFormatException e) {

```

```

        System.out.println(">> Valor inválido! Digite um número inteiro.");
    }
    break;

    case 4:
        System.out.println("\nVisualização da Árvore AVL (Chave [FB]):");
        avl.mostrar();
        break;

    case 5:
        executarExemploEspecificacao();
        avl = new ArvoreAVL();
        int[] nos = {10, 8, 15, 2, 22, 25};
        for (int n : nos) avl.inserir(n);
        break;

    case 6:
        executarExercicioSlide38();
        break;

    case 7:
        executarExercicioSlide42();
        break;

    case 8:
        avl = new ArvoreAVL();
        System.out.println(">> Árvore reiniciada com sucesso! (Vazia)");
        break;

    case 0:
        System.out.println(">> Encerrando o programa. Até logo!");
        break;

    default:
        System.out.println(">> Opção inválida. Escolha entre 0 e 8.");
    }
}

scanner.close();
}

/**
 * Demonstração passo a passo do exemplo fornecido na especificação:
 * Inserção de 10, 5, 15, 2, 8, 22 -> Inserir 25 com RES em 15 -> Remover 5.
 */
private static void executarExemploEspecificacao() {

    System.out.println("\n=====
    ==");
    System.out.println("          DEMONSTRAÇÃO DO EXEMPLO DA ESPECIFICAÇÃO DA TAREFA          ");

    System.out.println("=====
    ==");

    ArvoreAVL arvore = new ArvoreAVL();
    int[] chavesIniciais = {10, 5, 15, 2, 8, 22};

    System.out.println("\n[PASSO 1] Inserindo chaves iniciais: 10, 5, 15, 2, 8, 22...");
    for (int c : chavesIniciais) {
        arvore.inserir(c);
    }

    System.out.println("\nÁrvore resultante após inserções iniciais:");

```

```

arvore.mostrar();

System.out.println("\n[PASSO 2] Inserindo chave 25...");
System.out.println("Ao inserir 25 na subárvore direita de 22 (que é filho direito de 15):");
System.out.println(" - 22 passa para FB = -1");
System.out.println(" - 15 passa para FB = -2 (DESBALANCEAMENTO detectado à direita)");
System.out.println("\nEstado conceitual antes da rotação:");
System.out.println("                10[0]");
System.out.println("           5[0]      15[-2]");
System.out.println("    2[0]      8[0]      22[-1]");
System.out.println("                25[0]");

System.out.println("\nComo FB(15) = -2 e FB(22) = -1 (FB <= 0 na subárvore direita):");
System.out.println(">> Rotação Esquerda Simples (RES) aplicada no nó 15.");
System.out.println("Fórmulas do Slide do Prof. Robinson Alves:");
System.out.println(" FB_B_novo = FB_B + 1 - min(FB_A, 0) = -2 + 1 - (-1) = 0");
System.out.println(" FB_A_novo = FB_A + 1 + max(FB_B_novo, 0) = -1 + 1 + 0 = 0");

arvore.inserir(25);
System.out.println("\nÁrvore após rotação RES em 15:");
arvore.mostrar();

System.out.println("\n[PASSO 3] Removendo chave 5...");
System.out.println("Detalhe da implementação:");
System.out.println(" 1. O nó 5 possui dois filhos (2 à esquerda e 8 à direita).");
System.out.println(" 2. Localiza o sucessor in-order: o menor elemento da subárvore direita (nó 8).");
System.out.println(" 3. Copia a chave 8 para a posição do 5.");
System.out.println(" 4. Remove o nó folha original 8 da subárvore direita.");
System.out.println(" 5. A subárvore direita de 8 diminuiu. Recalcula FB(8):");
System.out.println("    FB(8) = he(2) - hd(vazio) = 1 - 0 = +1.");
System.out.println(" 6. Regra do slide do Prof. Robinson Alves na remoção:");
System.out.println("    'Se FB(Vantecessor) != 0 pare'.");
System.out.println("    Como FB(8) = +1 != 0, a alteração de altura NÃO se propaga para a raiz 10 (FB(10)");
continua 0).");

arvore.remove(5);
System.out.println("\nÁrvore resultante após remover 5:");
arvore.mostrar();

System.out.println("\n=====
==");
}

/**
 * Exercício do Slide 38 do Prof. Robinson Alves:
 * Inserir na árvore AVL inicialmente vazia os seguintes elementos:
 * 10, 20, 30, 40, 50, 25, 60, 70, 80 e 90.
 */
private static void executarExercicioSlide38() {

System.out.println("\n=====
==");
System.out.println("          EXERCÍCIO DO SLIDE 38 - PROF. ROBINSON ALVES          ");
System.out.println("  Inserção sucessiva: 10, 20, 30, 40, 50, 25, 60, 70, 80 e 90  ");

System.out.println("\n=====
==");

ArvoreAVL arv38 = new ArvoreAVL();
int[] valores = {10, 20, 30, 40, 50, 25, 60, 70, 80, 90};

for (int v : valores) {
    System.out.println("\n>> Inserindo: " + v);

```

```

        arv38.inserir(v);
        arv38.mostrar();
    }
    System.out.println("\n>> Árvore final do Slide 38 balanceada com sucesso!");
}

/**
 * Exercício do Slide 42 do Prof. Robinson Alves:
 * Remover na árvore: 40, 25, 50, 10, 35, 30, 20, 70 e 60.
 */
private static void executarExercicioSlide42() {

    System.out.println("\n=====
    ==");
    System.out.println("          EXERCÍCIO DO SLIDE 42 - PROF. ROBINSON ALVES          ");

    System.out.println("=====
    ==");

    ArvoreAVL arv42 = new ArvoreAVL();
    // Árvore do Slide 42 tem os nós: 30, 20, 10, 50, 25, 40, 35, 70, 60, 80
    int[] base = {30, 20, 10, 50, 25, 40, 35, 70, 60, 80};
    for (int n : base) arv42.inserir(n);

    System.out.println("Árvore inicial do Slide 42:");
    arv42.mostrar();

    int[] remocoes = {40, 25, 50, 10, 35, 30, 20, 70, 60};
    for (int r : remocoes) {
        System.out.println("\n>> Removendo: " + r);
        arv42.remover(r);
        arv42.mostrar();
    }
    System.out.println("\n>> Remoções do Slide 42 executadas com sucesso!");
}
}

```

## 4. Saída dos Testes de Execução

Abaixo constam os registros de execução obtidos no terminal pelo programa Main, cobrindo o exemplo da especificação e os exercícios do slide do Prof. Robinson Alves:

```

=====
        SISTEMA DE TESTES - RVORE AVL (HERANÇA DE ABP)
        Conforme Slides e Aulas do Prof. Robinson Alves
        Aluna: Larissa Samara
=====

[PASSO 1] Inserindo chaves iniciais: 10, 5, 15, 2, 8, 22...
rvore resultante após inserções iniciais:
          10[0]
        5[0]      15[-1]
      2[0]    8[0]      22[0]

[PASSO 2] Inserindo chave 25...
Estado antes da rotação:
          10[0]
        5[0]      15[-2]
      2[0]    8[0]      22[-1]
                  25[0]

>> Rotação Esquerda Simples (RES) aplicada no nó 15.
Fórmulas do Slide do Prof. Robinson Alves:
    FB_B_novo = FB_B + 1 - min(FB_A, 0) = -2 + 1 - (-1) = 0

```

$$FB\_A\_novo = FB\_A + 1 + \max(FB\_B\_novo, 0) = -1 + 1 + 0 = 0$$

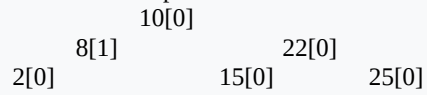
rvore após rotação RES em 15:



[PASSO 3] Removendo chave 5...

- Sucessor in-order localizado: 8
- Recálculo de  $FB(8) = hf(2) - hf(vazio) = 1 - 0 = +1$
- Regra **do** slide: Se  $FB(Vantecessor) \neq 0$  pare. Propagação cessa no nó 8.

rvore resultante após remover 5:



=====